

Transfer Learning in Guidebook Grounding with Human Feedback for Robot Manipulation

Anonymous Author(s)

Affiliation

Address

email

Abstract: Robots in new environments often encounter unseen appliances whose operation is documented in user manuals and guidebooks that contain rich procedural, semantic, and dynamics information that could accelerate robot learning. However, grounding this information remains challenging: robots must map guidebook entities to observations, convert instructions into actions, and interpret described dynamics to monitor progress and detect failures. We present BookWorm, a framework for learning robot manipulation tasks from guidebooks using on-demand human feedback that is distilled into transferable knowledge. Our key insight is that guidebooks provide structured priors not only for action generation but also for understanding expected state transitions, enabling more informed execution, failure detection, and targeted human feedback. When failures occur, BookWorm actively queries a human for corrective feedback, incorporates these corrections into its policy, and distills reusable knowledge that can be transferred to future appliances within the same category. BookWorm combines skill synthesis, symbol grounding, and a guidebook-conditioned policy-forward model that selects and parametrizes executable skills while predicting expected future states for failure detection, querying for human feedback only when grounding failures occur. Experiments on real-world appliance manipulation demonstrate that BookWorm more effectively leverages guidebook information than prior approaches and transfers acquired knowledge to novel devices, improving zero-shot performance and reducing the amount of human supervision required for adaptation.

Keywords: Learning from Manuals, Learning from Human Feedback, Transfer Learning

1 Introduction

Consider a housekeeper in an unfamiliar home tasked with cooking and cleaning using a variety of appliances. Faced with a previously unseen device, they would naturally consult its user manual rather than discover its functionality through trial-and-error. We envision robot housekeepers that similarly leverage guidebooks to rapidly learn new tasks. Given the vast corpus of existing manuals, repair guides, and operating procedures, guidebooks represent a rich and largely untapped source of knowledge for accelerating robot learning in new environments.

The central challenge is **grounding** guidebook knowledge in the robot’s perception, action, and state spaces. At the perceptual level, the robot must associate textual and diagrammatic entities in the guidebook, such as buttons, handles, and indicators, with objects in its observations. At the action level, it must translate high-level procedural instructions into executable robot motions. Beyond perception and control, the robot must also understand the dynamics described in the guidebook (e.g., that pressing a particular button should illuminate an indicator or open a compartment) in order to reason about task progress and detect failures during execution. While human feedback can help resolve these grounding failures, repeatedly requiring the same corrections for every new

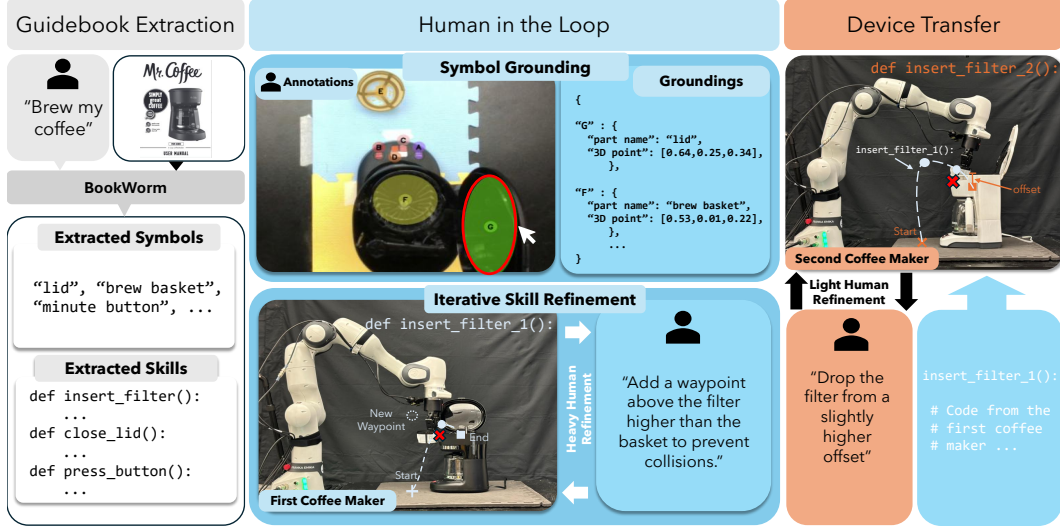


Figure 1: BookWorm leverages rich information from guidebooks to aid skill synthesis, planning, and dynamics understanding. To bridge action descriptions with low-level robot control, BookWorm actively queries humans for feedback and integrates it, updating our policy representation. We distill generalizable language feedback to improve transfer to new devices and guidebooks, reducing the amount of additional supervision needed to ground new guidebooks, tasks, and devices.

39 appliance is inefficient. To scale guidebook-driven robot learning, the robot must not only ground
 40 guidebook knowledge on a single device, but also transfer the knowledge acquired from previous
 41 devices to reduce supervision when adapting to new ones.

42 To address these challenges, we propose BookWorm, **Bootstrapping Object-grounded Operational**
 43 **Knowledge With On-demand Robot Mentoring**, a framework that initializes robot behavior from
 44 guidebooks and improves it through on-demand human feedback. Our key insight is that guidebooks
 45 provide structured priors not only for action generation, but also for understanding expected state
 46 transitions during task execution. BookWorm uses this information to identify grounding failures,
 47 query humans only when necessary, and distill the resulting corrections into reusable knowledge
 48 that can be transferred to future devices. As a result, the robot progressively improves its ability to
 49 ground new guidebooks while reducing the amount of additional human supervision required. Book-
 50 Worm integrates skill synthesis, symbol grounding, guidebook-conditioned forward prediction, and
 51 failure-driven human feedback, as shown in Figure 1. Rather than predicting raw robot trajectories
 52 directly, BookWorm represents behavior hierarchically: a guidebook-conditioned high-level policy
 53 selects skills and parameters, while grounded executable code skills realize these decisions as robot
 54 motions.

55 We evaluate BookWorm on three real-world household appliances in both single device settings and
 56 multi-device transfer settings. Our experiments demonstrate that guidebook grounding improves
 57 task performance by 45% points over the next most successful baseline, that guidebook-conditioned
 58 dynamics prediction improves state tracking and failure detection, and that distilled feedback enables
 59 more efficient adaptation to previously unseen devices.

60 Our contributions are threefold: 1) we introduce BookWorm, the first framework for learning real-
 61 world robot manipulation tasks from guidebooks through on-demand human feedback; 2) we pro-
 62 pose a guidebook-grounding framework that jointly reasons about procedural instructions, visual
 63 entities, and guidebook dynamics while actively soliciting multimodal human feedback to resolve
 64 grounding failures; 3) we show that distilling guidebook-grounded human feedback into reusable
 65 knowledge enables more efficient transfer to novel appliances while reducing the amount of required
 66 human supervision.

Robot learning from guidebooks and language specifications. Prior work has explored using manuals for **real-world robotic tasks**, primarily for spatial grounding to align diagrams or symbolic instructions to image observations [1, 2, 3]. ApBot [4] learns symbolic appliance models from manuals but focuses on simple button-centric interactions. Unlike our work, it does not synthesize skills from the guidebook or refine them with human feedback. In **simulated domains**, several works tackle sequential decision making from language guidelines. Prior methods learn policies from game manuals [5, 6], ground entities and dynamics in gridworld instructions [7, 8, 9, 10], or enforce language-specified constraints during RL [11, 12, 13, 14]. Unlike these settings, our problem requires grounding noisy real-world observations to procedural guidebooks where exhaustive RL exploration is infeasible. We instead treat the guidebook as a structured prior over perception, action, and task dynamics. Our setting also resembles rule-constrained real-world systems [15, 16, 17], but these works do not address procedural skill synthesis, actively-queried human feedback, or transferring relevant knowledge between domains.

Human-in-the-loop Robot Learning. Several works refine robot behavior from human feedback. DROC [18] distills iterative human corrections into code policies, while LYRA [19] expands a reusable skill library, and Vocal Sandbox [20] learns skills from multimodal feedback using code-based policies. Unlike these methods, which assume constant human supervision or focus on skill acquisition from feedback, we use guidebooks as the initial source of structured task knowledge and actively query humans only when grounding failures occur. Our query mechanism is related to opportunistic active learning [21, 22], active predicate learning [23], decision-theoretic query arbitration [24], and ConformalDAgger [25]. While these methods decide to query humans through uncertainty estimates over predefined symbols or classifiers, BookWorm uses detected mismatches between guidebook-conditioned next state predictions and observed outcomes as a signal to query for human feedback, allowing it to proactively address failures in perception, skill execution, or skill selection.

Continual Learning. Our transfer setting also differs from standard continual learning approaches that mitigate forgetting through parameter regularization [26], replay [27], modular sub-networks [28], or large pretrained VLAs [29, 30]. We do not study long task streams or catastrophic forgetting. Instead, we focus on forward transfer: how experience and human feedback from a source appliance can improve learning sample efficiency on a target appliance. Rather than preserving neural weights, we accumulate symbolic knowledge including executable skills, grounding-related corrections, and distilled language corrections. LOTUS [31] similarly builds reusable skill libraries from demonstrations, whereas our method seeds skills directly from guidebooks and transfers knowledge across appliances through language grounding.

Composable policies. Our method builds on representing policies as executable programs [32, 33, 34]. Code representations naturally support skill composition, parameterization, and language-based refinement. However, prior work largely studies common-sense pick-and-place tasks and does not address synthesizing policies from domain-specific procedural knowledge in guidebooks.

Language-conditioned dynamics and predictive robot models. Prior work has studied how language improves latent world models [35, 36, 37, 38] and dynamics models [39]. Another thread of work focuses on VLAs that jointly predict actions and next image observations [40, 41, 42, 43, 44, 45, 46]. Our work is inspired by principles of these works—BookWorm leverages language dynamics information in the guidebook to better understand the domain and task. Our policy jointly predicts actions and language next states, but does not undergo the computationally heavy prediction task of learning an explicit world model in image-space. By representing states and actions through semantically grounded language and code abstractions, our predicted next states robustly guide policy execution and inherently serve as out-of-distribution triggers for human feedback.

3 Problem Setup

We study guidebook-conditioned robot manipulation. For each appliance D , the robot is given a guidebook G and a task instruction l , such as brewing coffee or disabling a thermostat schedule. In this work, we focus on using the instructions, diagrams, and described state changes needed to execute manipulation tasks, leaving broader use of safety and troubleshooting information to future work.

At each timestep t , the robot receives an image observation I_t , a robot state s_t , and a textual interaction history h_t . The robot state includes proprioceptive information such as end-effector pose, joint positions, force measurements, and gripper state. The history h_t contains previous language descriptions of states, executed actions, and any human feedback received so far. We denote the task context by $c = (G, l)$. The goal is to complete the task specified by l on appliance D while minimizing the amount of human feedback required.

We model robot behavior hierarchically. The high-level policy selects a skill and its parameters,

$$\pi : I_t \times h_t \times c \times \Omega \mapsto (\omega_t, \theta_t),$$

where Ω is the current skill library, $\omega_t \in \Omega$ is the selected skill, and θ_t contains the parameters needed to instantiate that skill. This high-level skill-parameter pair is the action space of the decision-making problem studied in this paper. The robot state s_t is used by the selected skill implementation rather than by the high-level decision model.

The selected skill is then executed by a low-level code-based skill implementation. Each skill maps its parameters and the robot state to a sequence of executable robot motions,

$$\omega_t : \theta_t \times s_t \mapsto X_t,$$

where $X_t = \{x_{t,i}\}_{i=1}^k$ is a variable-length sequence of Cartesian end-effector poses. Thus, the high-level policy decides *what* skill to execute and with *which* parameters, while the low-level skill implementation determines *how* to realize that decision as robot motion.

Some skill parameters correspond to symbolic guidebook entities, such as buttons, handles, lids, indicators, or compartments. These parameters must be grounded before execution. We denote the set of parameters requiring spatial grounding by Θ_G . For each appliance D , the robot must construct a device-specific grounding map

$$\Gamma_D : \Theta_G \rightarrow \mathbb{R}^3,$$

which maps guidebook entities to 3D locations in the robot workspace. Since appliance geometry and layout vary across devices, these spatial coordinates are device-specific and must be estimated for each new appliance.

Human-in-the-loop. During training, the robot may query a human for feedback. The human can provide freeform language feedback l_t^H to correct behavior, or visual annotation feedback such as keypoints or polygonal regions to correct symbol grounding. The human only provides feedback when explicitly queried by the robot. During evaluation, no human feedback is available.

Single-task and Transfer Learning. We consider two learning settings. In the *single-device setting*, the robot learns to perform a task on one appliance D using its guidebook and queried human feedback. In the *transfer learning setting*, the robot first learns on a source appliance D_s and then adapts to a target appliance D_t from the same appliance category. The target appliance may differ in geometry, layout, and terminology, but often shares useful skills and corrective feedback with the source appliance. The objective is to more efficiently learn the task on D_t with less human feedback based on experience gained on D_s .

Transfer does not mean reusing fixed spatial coordinates from the source device. Instead, the target device is regrounded using its own guidebook and observations. The reusable knowledge consists of skill implementations, grounding-related corrections, and distilled human feedback that can guide skill synthesis, grounding, and high-level decision making on future appliances.

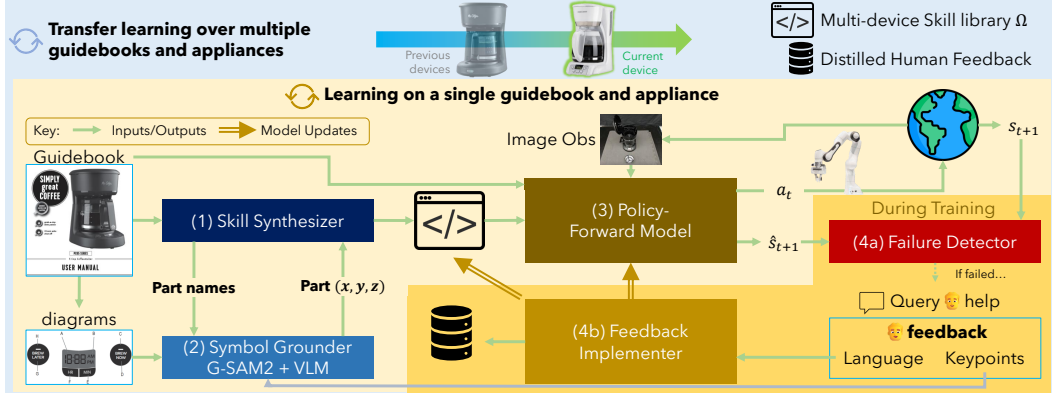


Figure 2: **System Overview.** BookWorm initializes a guidebook-grounded policy by synthesizing skills, grounding guidebook entities, and predicting expected next states. During training, it detects failures, queries humans for feedback, and distills corrections into reusable knowledge. When adapting to a new device, BookWorm reuses prior skills and distilled feedback while regrounding the target appliance from its own guidebook and observations.

159 4 BookWorm

160 BookWorm instantiates the hierarchical problem setup from Section 3 via a guidebook-grounded
 161 policy refined through human feedback and transferred across devices (see Figure 2). Given guide-
 162 book G and task instruction l , BookWorm builds an initial policy prior by synthesizing executable
 163 skills, identifying parameters requiring grounding, mapping guidebook entities to observations, and
 164 predicting expected state changes per action. When observed outcomes diverge from guidebook-
 165 conditioned expectations, BookWorm queries a human for corrective feedback and updates the pol-
 166 icy accordingly.

167 The key design principle is *reusable feedback*. Corrections fix current failures and are distilled into
 168 a knowledge base informing future skill synthesis, grounding, and execution. For new devices in
 169 the same category, prior skills and distilled feedback provide stronger priors for zero-shot execu-
 170 tion, while grounding remains device-specific. Skill synthesis, policy-forward prediction, failure
 171 detection, and feedback integration are all performed via an off-the-shelf vision-language model
 172 (GPT-5.4). Symbol grounding combines open-vocabulary segmentation with guidebook diagrams
 173 and human annotations.

174 4.1 Guidebook-Grounded Initialization

175 BookWorm converts the guidebook into a policy prior over skills, grounded parameters, and ex-
 176 pected state transitions. Skill library Ω is synthesized from task context $c = (G, l)$ and a robot API
 177 (e.g., `move_to_pose()`, `open_gripper()`), decomposing guidebook instructions into semantically
 178 meaningful code-based skills (opening a lid, inserting a filter, pressing a button). Code-based skills
 179 provide a structured interface between guidebook language and robot control: synthesizable from
 180 instructions, editable via language feedback, and reusable across devices.

181 To ground symbolic parameters, BookWorm constructs grounding map Γ_D mapping each $\theta_g \in \Theta_G$
 182 to a 3D workspace location. GSAM2 [47, 48, 49] runs on robot observations using queries from
 183 Θ_G , producing candidate masks M_G backprojected into point clouds. The symbol grounder then
 184 reasons over masks, guidebook diagrams G_d , and entities Θ_G to assign 3D locations. Diagrams
 185 disambiguate visually similar detections—e.g., buttons identical in appearance but distinguishable
 186 by guidebook ordering and labels. Unresolvable entities trigger a human query for keypoint or
 187 polygonal annotations.

188 The high-level policy selects skill-parameter pairs (ω_t, θ_t) , augmented with guidebook-conditioned
 189 forward prediction:

$$\pi_{\text{pf}} : I_t \times h_t \times c \times \Omega \rightarrow (\omega_t, \theta_t, \hat{l}_{s,t+1}),$$

190 where $\hat{l}_{s,t+1}$ describes the expected post-execution state. This connects guidebook dynamics to
 191 failure detection, making the guidebook both an action source and a semantic execution monitor.

192 4.2 Feedback-Driven Refinement

193 Guidebook-grounded initialization is imperfect: entities may be misgrounded, skills geometrically
 194 imprecise, or high-level selections incorrect. BookWorm treats human feedback as targeted correc-
 195 tion rather than continuous supervision.

196 After executing $a_t = \omega_t(\theta_t)$, a failure detector evaluates whether the observed transition matches
 197 the predicted one:

$$f_d : I_t \times I_{t+1} \times \hat{l}_{s,t+1} \rightarrow [0, 1],$$

198 returning failure probability p_{fail} . This single signal covers misgrounding, imprecise execution, and
 199 incorrect skill selection.

200 When p_{fail} exceeds the threshold (greater than 50% in our experiments), the robot queries the human
 201 for feedback l_t^H —ranging from skill-specific (“push the button harder”) to high-level (“press exit
 202 before changing the time”) to visual annotations for grounding failures. Representative examples of
 203 human feedback are provided in the appendix.

204 The feedback implementer applies corrections by scope: skill-specific feedback updates Ω ; visual
 205 annotations update Γ_D ; general feedback enriches the policy context. In parallel, corrections are
 206 distilled into knowledge base D_H , separating device-specific from transferable knowledge.

207 4.3 Transfer Across Devices

208 When adapting to a new appliance, BookWorm reuses knowledge from prior appliances in the same
 209 category. Despite differences in geometry, layout, and terminology, appliances often share task
 210 structure, manipulation strategies, and relevant feedback.

211 Given target appliance D_t , BookWorm reuses the high-level policy and initializes Ω from both the
 212 new guidebook and D_H . Prior skills are reused directly, adapted when geometry differs, or replaced
 213 with new variants as needed. The target is grounded from its own guidebook and observations,
 214 yielding Γ_{D_t} . During execution, the policy selects among skill variants based on the current guide-
 215 book, observations, and interaction history, improving zero-shot performance and reducing required
 216 human feedback.

217 5 Experiments

218 5.1 Experimental Setup

219 **Tasks:** We evaluate our method on a real-world Franka Emika
 220 Panda Arm in both the *single task* setting on a single device, and
 221 the *transfer learning* setting on multiple devices. Guidebooks are
 222 downloaded as PDFs, with only the English-language pages pro-
 223 cessed for efficiency. For the *single task* setting, we evaluate on a
 224 **Honeywell thermostat** (X2P RTH20B) where the task is to disable
 225 schedule settings by navigating multiple menus, and a **Mr. Cof-**
 226 **fee maker** (PC05) to load a filter, brew coffee, and adjust the time.
 227 To evaluate multi-device *transfer learning*, we first train on PC05,
 228 then transfer the learned policy to a novel, morphologically distinct
 229 **Black-and-Decker coffee maker** (CM1160-W). All tasks involve
 230 four discrete skill steps. See Figure 3 for our experimental appliances.



Figure 3: Experimental Appliances.

Training and Evaluation Procedure: For each method on each device, we train for five trials, where the human can be queried for corrective natural language feedback, then evaluate for ten trials. After providing feedback, the human operator manually resets the scene if an irrecoverable failure has occurred, allowing the trial to resume. During evaluation, all modules are frozen and operate without human intervention. With no human to query, the failure detector is inactive, though the policy still predicts the next state, which conditions its skill and parameter selection. All trials have a maximum horizon of 12 timesteps, and all methods emit an empty action to terminate the trial.

Metrics: We evaluate three key metrics: (1) full task success, requiring all steps in the correct sequence; (2) step completion rate, the average proportion of consecutive steps completed (out of 4 steps per task); and (3) number of human feedback rounds, to quantify human training effort.

Baselines: In the *single task setting*, we compare against **Code-as-Policies (CaP)** [32] and **Promptbook** [34]. CaP directly creates skill functions from a guidebook but does not explicitly ground action or dynamics information from the guidebook, nor solicit human feedback. Promptbook is similar to CaP but adheres to more concrete prompt guidelines and can query for human feedback. In the *transfer learning setting*, we compare against **Experience Replay** [27], which is provided the same human feedback as our policy. All baselines are provided the same observations as our method.

Ablations: In the *single-task setting*, we test four ablations of our method: not using the guidebook for planning or skill synthesis (**No Guidebook**), not predicting the next state with the policy (**No next state pred**), and not having human language feedback (**No human feedback**). In the *transfer learning setting*, we ablate the usage of distilled human feedback and refined skills by training everything from scratch on the new coffee maker (**Ours, Single Device**).

5.2 Experimental Results

To evaluate our framework, we seek to answer three research questions: (RQ1) Does our method effectively implement and execute actions from guidebooks? (RQ2) Does our method effectively utilize dynamics information from guidebooks? (RQ3) Can guidebook-grounded knowledge acquired from previous devices be leveraged to more efficiently learn on new devices? We test RQ1 and RQ2 in the single-task, single-device setting, and RQ3 in the transfer learning, two-device setting.

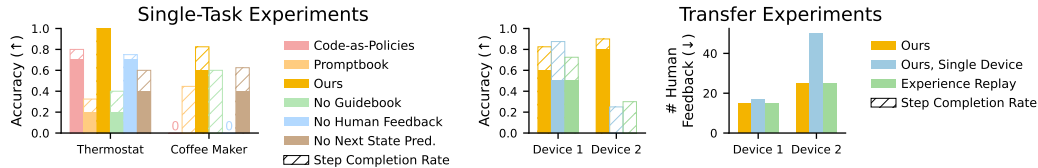


Figure 4: **Left:** In the single-task setting, our method outperforms prior work baselines and all ablations on both devices. **Right:** In the transfer learning setting, our method achieves comparable success as ablations and baselines when learning the first device, but far outperforms them when transferred to the second device, learning with less human effort than a non-transfer learning ablation learned solely on the second device.

Figure 4 depicts our single-task and multi-device transfer learning experimental results. Each bar represents 10 evaluation trials. Solid bars represent full task success, and diagonal hollow bars represent the step completion rate. Below, we answer each research question.

RQ1: Skill execution and implementation from guidebooks.

Removing the guidebook from the policy and skill synthesizer (**No Guidebook**) decreases the average single-task success rate of BookWorm from 80% to 10%, confirming that guidebook grounding is necessary for both skill synthesis and sequential planning. However, the guidebook alone is not

sufficient for enabling performant policies. We also demonstrate the importance of soliciting human feedback for refining our guidebook-initiated policies. Our method without human feedback only averages 35% over the two devices.

We also compare to prior work CaP and Promptbook to measure how effectively they can create policies grounded to guidebooks. CaP and Promptbook achieve 35% and 10% respectively, far lower than ours.

RQ2: Guidebook Dynamics for State Tracking and Failure Detection.

In BookWorm, the policy jointly predicts the action and the resulting next state in language. Removing next-state prediction drops performance from 80% to 40%, demonstrating that our method actively grounds guidebook dynamics in real-world transitions, especially when visual feedback is ambiguous.

RQ3: Transfer Learning Across Devices.

In the transfer learning setting, our method adapts more efficiently, with fewer rounds of human feedback, to the new appliance than all other methods tested. Our method performs 45% points better than the **Experience Replay** baseline when averaged over both devices, despite using the same amount of human feedback, further demonstrating our ability to extract generalizable information from prior devices’ guidebooks when transferring to new devices. Compared to our single-device ablation that does not transfer knowledge between devices, our method achieves a 45% points higher success rate on average with 31 fewer rounds of human feedback. This demonstrates that our transfer mechanism, consisting of distilling generalizable language feedback and refining skill functions from the previous device, significantly improves transfer to new devices within the same appliance class while reducing device-specific human supervision.

6 Conclusion

We presented BookWorm, a framework for learning real-world robot manipulation tasks from guidebooks by grounding guidebook entities, synthesizing executable skills, and using targeted human feedback to refine and transfer the resulting policy. We empirically showed that our method better utilizes guidebook information than prior works, achieves higher performance with less human effort in a transfer learning setting. We believe BookWorm opens the door to leveraging the large corpus of human-written domain information to help robots more efficiently learn physical tasks in real households, and that it can substantially improve sample efficiency over robot learning frameworks that do not use guidebooks.

Limitations and Future Work. We experimented with only code-based skill representations. While this offers an efficient skill representation easily extracted from language guidebooks and refined with language feedback, it can be limited for articulated appliances where precise waypoints and end-effector orientations are needed. Future work can examine having a hybrid skill library of code-based and learned policy heads to further bridge the gap between language from guidebooks and low-level control. Actions like pick-and-place can be easily represented as code-based skills, while precise and contact-rich insertion tasks may require RL-trained skills.

BookWorm considers how guidebook information can influence skill synthesis, planning, and dynamics understanding. Future work can go further in exploiting guidebook information, such as to learn symbolic reward functions that can help keep track of the policy’s progress and aid in better action prediction during evaluation, or in initializing safety-oriented constraints to help robots avoid hazardous situations described in guidebooks.

References

- [1] C. Tie, S. Sun, J. Zhu, Y. Liu, J. Guo, Y. Hu, H. Chen, J. Chen, R. Wu, and L. Shao. Manual2skill: Learning to read manuals and acquire robotic skills for furniture assembly using vision-language models. *arXiv preprint arXiv:2502.10090*, 2025.
- [2] C. Tie, S. Sun, Y. Lin, Y. Wang, Z. Li, Z. Zhong, J. Zhu, Y. Pang, H. Chen, J. Chen, et al. Manual2skill++: Connector-aware general robotic assembly from instruction manuals via vision-language models. *arXiv preprint arXiv:2510.16344*, 2025.
- [3] J. Zhang, A. Cherian, C. Rodriguez, W. Deng, and S. Gould. Manual-pa: Learning 3d part assembly from instruction diagrams. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 6304–6314, 2025.
- [4] J. Zhang, H. Zhang, A. Xiao, and D. Hsu. Robot operation of home appliances by reading user manuals. *arXiv preprint arXiv:2505.20424*, 2025.
- [5] S. Branavan, D. Silver, and R. Barzilay. Learning to win by reading manuals in a monte-carlo framework. *Journal of Artificial Intelligence Research*, 43:661–704, 2012.
- [6] Y. Wu, Y. Fan, P. P. Liang, A. Azaria, Y. Li, and T. M. Mitchell. Read and reap the rewards: Learning to play atari with the help of instruction manuals. *Advances in Neural Information Processing Systems*, 36:1009–1023, 2023.
- [7] A. W. Hanjie, V. Y. Zhong, and K. Narasimhan. Grounding language to entities and dynamics for generalization in reinforcement learning. In *International conference on machine learning*, pages 4051–4062. PMLR, 2021.
- [8] K. Narasimhan, R. Barzilay, and T. Jaakkola. Grounding language for transfer in deep reinforcement learning. *Journal of Artificial Intelligence Research*, 63:849–874, 2018.
- [9] A. Nguyen and S. Lee. Language-conditioned world model improves policy generalization by reading environmental descriptions. *arXiv preprint arXiv:2511.22904*, 2025.
- [10] V. Zhong, T. Rocktäschel, and E. Grefenstette. Rtfm: Generalising to novel environment dynamics via reading. *arXiv preprint arXiv:1910.08210*, 2019.
- [11] T.-Y. Yang, M. Y. Hu, Y. Chow, P. J. Ramadge, and K. Narasimhan. Safe reinforcement learning with natural language constraints. *Advances in Neural Information Processing Systems*, 34:13794–13808, 2021.
- [12] Z. Wang, M. Fang, T. Tomilin, F. Fang, and Y. Du. Safe multi-agent reinforcement learning with natural language constraints. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 38007–38015, 2026.
- [13] B. Prakash, N. R. Waytowich, A. Ganesan, T. Oates, and T. Mohsenin. Guiding safe reinforcement learning policies using structured language constraints. In *SafeAI@ AAAI*, pages 153–161, 2020.
- [14] X. Lou, J. Zhang, Z. Wang, K. Huang, and Y. Du. Safe reinforcement learning with free-form natural language constraints and pre-trained language models. *arXiv preprint arXiv:2401.07553*, 2024.
- [15] Y. Bai, S. Kadavath, S. Kundu, A. Askill, J. Kernion, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon, et al. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073*, 2022.
- [16] A. Censi, K. Slutsky, T. Wongpiromsarn, D. Yershov, S. Pendleton, J. Fu, and E. Frazzoli. Liability, ethics, and culture-aware behavior specification using rulebooks. In *2019 international conference on robotics and automation (ICRA)*, pages 8536–8542. IEEE, 2019.

- [17] M. F. Ginting, D.-K. Kim, S.-K. Kim, B. J. Krishna, M. J. Kochenderfer, S. Omidshafiei, and A.-a. Agha-mohammadi. Saycomply: Grounding field robotic tasks in operational compliance through retrieval-based language models. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 13730–13736. IEEE, 2025.
- [18] L. Zha, Y. Cui, L.-H. Lin, M. Kwon, M. G. Arenas, A. Zeng, F. Xia, and D. Sadigh. Distilling and retrieving generalizable knowledge for robot manipulation via language corrections. In *2024 IEEE international conference on robotics and automation (ICRA)*, pages 15172–15179. IEEE, 2024.
- [19] Y. Meng, Z. Sun, M. Fest, X. Li, Z. Bing, and A. Knoll. Growing with your embodied agent: A human-in-the-loop lifelong code generation framework for long-horizon manipulation skills. *arXiv preprint arXiv:2509.18597*, 2025.
- [20] J. Grannen, S. Karamcheti, S. Mirchandani, P. Liang, and D. Sadigh. Vocal sandbox: Continual learning and adaptation for situated human-robot collaboration. *arXiv preprint arXiv:2411.02599*, 2024.
- [21] J. Thomason, A. Padmakumar, J. Sinapov, J. Hart, P. Stone, and R. J. Mooney. Opportunistic active learning for grounding natural language descriptions. In *Conference on robot learning*, pages 67–76. PMLR, 2017.
- [22] A. Padmakumar, P. Stone, and R. Mooney. Learning a policy for opportunistic active learning. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 1347–1357, 2018.
- [23] A. Li and T. Silver. Embodied active learning of relational state abstractions for bilevel planning. *arXiv preprint arXiv:2303.04912*, 2023.
- [24] K. Bullard, A. L. Thomaz, and S. Chernova. Towards intelligent arbitration of diverse active learning queries. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 6049–6056. IEEE, 2018.
- [25] M. Zhao, H. Admoni, R. Simmons, A. Ramdas, and A. Bajcsy. Conformalized interactive imitation learning: Handling expert shift and intermittent feedback. In *International Conference on Learning Representations*, volume 2025, pages 9238–9251, 2025.
- [26] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526, 2017.
- [27] D. Rolnick, A. Ahuja, J. Schwarz, T. Lillicrap, and G. Wayne. Experience replay for continual learning. *Advances in neural information processing systems*, 32, 2019.
- [28] A. A. Rusu, N. C. Rabinowitz, G. Desjardins, H. Soyer, J. Kirkpatrick, K. Kavukcuoglu, R. Pascanu, and R. Hadsell. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016.
- [29] J. Hu, J. Shim, C. Tang, Y. Sung, B. Liu, P. Stone, and R. Martin-Martin. Simple recipe works: Vision-language-action models are natural continual learners with reinforcement learning. *arXiv preprint arXiv:2603.11653*, 2026.
- [30] H. Liu, C. Kim, B. Liu, M. Liu, and Y. Zhu. Pretrained vision-language-action models are surprisingly resistant to forgetting in continual learning. *arXiv preprint arXiv:2603.03818*, 2026.
- [31] W. Wan, Y. Zhu, R. Shah, and Y. Zhu. Lotus: Continual imitation learning for robot manipulation through unsupervised skill discovery. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 537–544. IEEE, 2024.

- [32] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng. Code as policies: Language model programs for embodied control. In *arXiv preprint arXiv:2209.07753*, 2022.
- [33] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg. Progprompt: Generating situated robot task plans using large language models. *arXiv preprint arXiv:2209.11302*, 2022.
- [34] M. G. Arenas, T. Xiao, S. Singh, V. Jain, A. Ren, Q. Vuong, J. Varley, A. Herzog, I. Leal, S. Kirmani, et al. How to prompt your robot: A promptbook for manipulation skills with code as policies. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4340–4348. IEEE, 2024.
- [35] D. Chen, T. Moutakanni, W. Chung, Y. Bang, Z. Ji, A. Bolourchi, and P. Fung. Planning with reasoning using vision language world model. *arXiv preprint arXiv:2509.02722*, 2025.
- [36] J. Gkountouras, M. Lindemann, P. Lippe, E. Gavves, and I. Titov. Language agents meet causality—bridging llms and causal world models. In *International Conference on Learning Representations*, volume 2025, pages 37311–37336, 2025.
- [37] A. Zhang, K. Nguyen, J. Tuyls, A. Lin, and K. Narasimhan. Language-guided world models: A model-based approach to ai control. In *Proceedings of the 4th Workshop on Spatial Language Understanding and Grounded Communication for Robotics (SpLU-RoboNLP 2024)*, pages 1–16, 2024.
- [38] N. Dainese, P. Marttinen, and A. Ilin. Reader: Model-based language-instructed reinforcement learning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 16583–16599, 2023.
- [39] J. Lin, Y. Du, O. Watkins, D. Hafner, P. Abbeel, D. Klein, and A. Dragan. Learning to model the world with language. *arXiv preprint arXiv:2308.01399*, 2023.
- [40] H. Wu, Y. Jing, C. Cheang, G. Chen, J. Xu, X. Li, M. Liu, H. Li, and T. Kong. Unleashing large-scale video generative pre-training for visual robot manipulation. In *International Conference on Learning Representations*, volume 2024, pages 10641–10662, 2024.
- [41] J. Cen, S. Huang, Y. Yuan, K. Li, H. Yuan, C. Yu, Y. Jiang, J. Guo, X. Li, H. Luo, et al. Rynnvla-002: A unified vision-language-action and world model. *arXiv preprint arXiv:2511.17502*, 2025.
- [42] C. Zhu, R. Yu, S. Feng, B. Burchfiel, P. Shah, and A. Gupta. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets. *arXiv preprint arXiv:2504.02792*, 2025.
- [43] J. Zhang, Y. Guo, Y. Hu, X. Chen, X. Zhu, and J. Chen. Up-vla: A unified understanding and prediction model for embodied agent. *arXiv preprint arXiv:2501.18867*, 2025.
- [44] Y. Shen, F. Wei, Z. Du, Y. Liang, Y. Lu, J. Yang, N. Zheng, and B. Guo. Videovla: Video generators can be generalizable robot manipulators. *Advances in neural information processing systems*, 38:95597–95621, 2026.
- [45] Y. Tian, S. Yang, J. Zeng, P. Wang, D. Lin, H. Dong, and J. Pang. Predictive inverse dynamics models are scalable learners for robotic manipulation. In *International Conference on Learning Representations*, volume 2025, pages 92033–92052, 2025.
- [46] K. Wang, P. Zhang, Z. Wang, Y. Gao, L. Li, Q. Wang, H. Chen, Y. Lu, Z. Yang, L. Wang, et al. Vagen: Reinforcing world model reasoning for multi-turn vlm agents. *Advances in Neural Information Processing Systems*, 38:172871–172933, 2026.

- 443 [47] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, C. Li, J. Yang, H. Su, J. Zhu, and L. Zhang.
444 Grounding dino: Marrying dino with grounded pre-training for open-set object detection.
445 *ECCV*, 2024. URL <https://arxiv.org/abs/2303.05499>.
- 446 [48] T. Ren, Q. Jiang, S. Liu, Z. Zeng, W. Liu, H. Gao, H. Huang, Z. Ma, X. Jiang, Y. Chen,
447 Y. Xiong, H. Zhang, F. Li, P. Tang, K. Yu, and L. Zhang. Grounding dino 1.5: Advance the
448 "edge" of open-set object detection, 2024.
- 449 [49] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland,
450 L. Gustafson, E. Mintun, J. Pan, K. V. Alwala, N. Carion, C.-Y. Wu, R. Girshick, P. Dollár,
451 and C. Feichtenhofer. Sam 2: Segment anything in images and videos, 2024. URL <https://arxiv.org/abs/2408.00714>.
452

453 A Method Details

454 We describe the modules of BookWorm in more detail. Each module is implemented as an off-the-
455 shelf call to GPT-5.4. Prompt templates can be viewed on our [project site](#).

456 **(1) Skill Synthesizer.** The skill synthesizer processes the guidebook and task description to create
457 a skill library Ω of skill functions ω . Concretely, it examines the task context c (which includes
458 the guidebook and task instruction; see Section 3), and the robot base API (containing basic func-
459 tions for the robot, such as `move_to_pose(xyz, euler)` and `open_gripper()`), and implements
460 from scratch a set of skill functions, each with a unique parameter space, which call the robot base
461 API. The key challenge for the skill synthesizer is two-fold. First, it must decompose the task into
462 discrete, reusable skill functions based on the guidebook’s procedural instructions. Second, it must
463 define the parameter space for each individual skill, aggregate them into a global parameter space
464 Θ , and extract the subset $\Theta_G \subseteq \Theta$ (e.g., buttons and device parts) that requires spatial grounding in
465 Cartesian coordinates.

466 **(2) Symbol Grounder.** Given Θ_G , the symbol grounder produces a mapping from each entity
467 $\theta_g \in \Theta_G$ to its (x, y, z) coordinates, spatially grounding each entity that must be manipulated
468 for the task. It runs GroundingDINO 2 [47, 48] on an image of the scene and language queries
469 representing Θ_G to get a set of candidate segmentation masks M_G (which are backprojected into
470 Cartesian-space point clouds). It then processes guidebook diagrams G_d , alongside M_G and Θ_G ,
471 to produce a 1:1 map $\theta_g \mapsto (x, y, z)$. To ground non-segmentable entities, a human is queried for
472 keypoint or polygonal annotations.

473 **(3) Policy.** The VLM-based policy π takes in the image observation I_t , episode textual history
474 h_t , task context $c = (G, l)$, and skill library Ω . From this, it jointly predicts the action $\omega_t(\theta_t)$
475 and a language description of the expected next state, $l_{\hat{s}_{t+1}}$, if $\omega_t(\theta_t)$ were executed successfully.
476 We find that augmenting the policy as a joint forward model enables our framework to internalize
477 dynamics information provided in the guidebook, improving its action predictions and enabling our
478 downstream failure handler to decide when to proactively ask for human assistance.

479 **(4) Failure Handler.** To enable human-in-the-loop learning, our method needs to query for human
480 feedback and update its policy representations to implement this feedback. The failure handler
481 performs both functions with a failure detector and human feedback implementer. After the robot
482 executes the predicted action $a_t = \omega_t(\theta_t)$ from the policy, we record the transition (s_t, a_t, s_{t+1}) .
483 The failure detector $f_d : I_t \times I_{t+1} \times l_{\hat{s}_{t+1}} \mapsto [0, 1]$ takes in the before-and-after images of this
484 transition (I_t, I_{t+1}) and the guidebook-conditioned policy’s predicted next state, $l_{\hat{s}_{t+1}}$, and returns
485 the failure probability p_{fail} of action a_t . If p_{fail} is high, the failure handler queries the human
486 for freeform language feedback, l_{H_t} , which can range from the skill-level (e.g., “push the button
487 slightly harder”) to high-level general feedback (e.g., “press exit on unfamiliar screens”). Given l_{H_t} ,
488 the human feedback implementer either updates the skill library by calling the skill synthesizer to
489 update a specific skill ω , or updates the policy π by adding l_{H_t} to the episode history h_t . Human
490 feedback is selectively distilled and sorted into a textual knowledge base $\mathcal{D}_H$ of generalizable meta-
491 knowledge and device-specific knowledge, where it can be referenced to accelerate learning on new
492 devices in the outer learning loop.

493 B Task Details

494 **Thermostat.** Task: Turn off the schedule settings.

495 The initial condition of the scene is the thermostat somewhere on the workspace, placed in an axis-
496 aligned pose with the screen right-side-up for the wrist camera. The schedule setting starts “on,” and
497 the thermostat is on the home screen and idle (the screen is not lit).

498 The goal condition of the scene is that the schedule setting is turned to “off,” and the robot has
499 navigated back to the home screen on the thermostat.

500 A working sequence of actions that completes this task is:

- 501 1. Press left middle button (“Menu”)
- 502 2. Press left bottom button (“Select”)
- 503 3. Press right down button (Toggles the setting from on to off.)
- 504 4. Press left top button twice (“Save & Exit”, “Exit”)

505 Common failure modes of our policy encountered during training include pressing the button too
506 lightly so that it does not advance the screen, pressing the button too hard so that a single press
507 registers as multiple unintended presses, and pressing a button after the screen has timed out. The
508 thermostat has various timeouts for each screen. Since our VLM-based policy takes several seconds
509 to act, many failures involve the screen already resetting back to the home screen between the time
510 the image observation is captured and the time when the skill is executed in the world.

511 **Mr. Coffee Maker.** Task: Brew a cup of coffee and adjust the time.

512 The initial condition of the scene is the coffee maker is near the center region of the workspace, with
513 its buttons visible to the overhead camera. The coffee maker is plugged in with its lid open, and the
514 reusable filter is placed a few inches in front and to the right of the console on the coffee machine.
515 Both the brew now and brew later buttons are off (not illuminated).

516 The goal condition of the scene is that the reusable filter has been placed inside the brew basket with
517 the lid closed. The time has advanced at least two hours from the time in the initial condition, and
518 the brew now button is illuminated.

519 A working sequence of actions that completes this task is:

- 520 1. Insert the coffee filter into the brew basket.
- 521 2. Close the lid.
- 522 3. Press the Brew Now button.
- 523 4. Press the hour button twice.

524 Common failure modes of the policy include spatial imprecisions when pressing buttons, and not
525 knowing when to terminate the policy—it would often continue pressing the hour button or the brew
526 now buttons even after the task were complete, sometimes undoing its own progress and turning off
527 the coffee maker brewing in the process.

528 **Black-and-Decker Coffee Maker.** Task: Brew a cup of coffee and adjust the time.

529 The initial and goal conditions of this task and device are extremely similar to that of **Mr. Coffee**
530 **Maker**, since this device was only used in our multi-device transfer experiments. The main differ-
531 ence with **Mr. Coffee Maker** is that the illumination after pressing the button that begins brewing
532 is in the screen, instead of on the button.

533 A working sequence of actions that completes this task is:

- 534 1. Insert the coffee filter into the brew basket.
- 535 2. Close the lid.
- 536 3. Press the On/off button.
- 537 4. Press the hour button twice.

538 The main failure mode of our policy on this task involves the step of closing the lid, which is very
539 sensitive to the position of the coffee maker. Slightly variations can cause IK to fail to reach the
540 prescribed waypoints in the skill function, causing the coffee maker’s position to shift while closing
541 the lid. Like the other appliances, another failure mode arose from spatial imprecision in pressing
542 buttons.

C Human Feedback Examples

We list some representative examples of human feedback on our three sets of experiments.

Thermostat. Task: Turn off the schedule settings.

- “When encountering unfamiliar screens, press ‘save and exit’ or ‘back’ to return to a more familiar screen.”
- “When button pressing, press 20% harder.”
- “Move 0.5cm in the +y direction when pressing the top left button.”

Mr. Coffee Maker. Task: Brew a cup of coffee and adjust the time.

- “Try placing the filter into the brew basket again. You hit the coffee maker when trying to put the coffee filter in, leading to it not getting in. Try to get the filter higher (+z direction) before putting the filter in.”
- “To close the lid, do not open then close your gripper. The lid does not open about the y axis, it opens about the x axis. Keep the gripper closed, move to a waypoint to the left of the lid (-y direction), then to a waypoint to the right of the lid (+y direction).”
- “Move 0.7cm more forward when pressing buttons.”

Black-and-Decker Coffee Maker. Task: Brew a cup of coffee and adjust the time.

- “Open the gripper after the robot reaches the waypoint right above the brew basket. Do not move to a waypoint lower inside the brew basket.”
- “When closing the lid, increase your last two waypoints by roughly 5cm in height.”
- “Change your button pressing offset to be 1.5cm further to the left.”

D Baseline Implementation Details

D.1 Single-Device Baselines

Code-as-Policies [32]. At the beginning of each episode, Code-as-Policies creates a new skill library in python. At each timestep, Code-as-Policies predicts the skill-parameter call as the action to execute in the world, and an optional function redefinition. If Code-as-Policies outputs this, then that function in its skill library is redefined for all future timesteps in the episode. In keeping with the spirit of the original paper, Code-as-Policies does not support implementing human feedback during training because it is not a human-in-the-loop learning framework.

PromptBook [34]. The key differences between Code-as-Policies and Promptbook are the 7 specific ingredients in the prompt of Promptbook and its access to human feedback. We allow Promptbook to both optionally redefine any function in its skill library, and also query for human help whenever it believes it is not making enough progress. At the end of each episode, the human has the opportunity to provide episode-level feedback to Promptbook, which is integrated in the context of the next episode. Because Promptbook supports human refinement, we allow Promptbook to maintain the same, continually updating skill functions over each episode, similar to our method.

D.2 Multi-Device Baseline

Experience Replay. Experience replay is a simple and popular approach from the continual learning literature. In continual learning, it involves co-training with data from older tasks to prevent catastrophic forgetting. In our experiments, we implemented experience replay as an additional section in the context of the policy that is a collection of all human feedback for the skill functions and policy, that our method was trained on during its 5 training episodes for both devices. Thus, experience replay receives the same language feedback as our approach, but it does not have the

585 iterative human-in-the-loop updating, triggered by our framework’s failure detector, that is key to
586 our method’s success.